A PRODUCT MANAGER'S FIELD GUIDE

Rationale

Turn customer feedback into what to build next.

Paste customer feedback. Get a ranked, evidence-backed answer to "what should we build next?" and a spec you can trust.

What it is, how it works, why it is built this way, and how to talk about it in an interview.

Moulshree Bhutra · concept project · November 2026

INSIDE THIS GUIDE

What you will learn

01 The 60-second version

02 The problem, in plain words

03 A walk through the product

04 Under the hood: how it works

05 How it reads feedback: grouping

06 The priority score

07 Writing the spec you can trust

08 Safety, privacy and abuse protection

09 Product decisions and trade-offs

10 Metrics, quality and testing

11 Interview cheat sheet

12 Glossary and code map

How to read this: every chapter opens with the plain-English idea, then shows how it really works, then tells you what a PM should take from it. Coloured boxes mark analogies, PM lenses and warnings. You can read it front to back in about an hour, or jump to chapter 11 (cheat sheet) right before an interview.

CHAPTER 01 · START HERE

The 60-second version

Rationale is a small web app that takes a messy pile of customer feedback and gives a product manager back three things: a ranked list of what to build, the evidence behind each item, and a one-page spec for the one they pick.

3 steps, one screen each	**1 line** the whole scoring formula	**0** invented customers or figures allowed in a spec

Who it is for, and what job it does

Picture a PM at a B2B software company. Every week they get feedback from sales calls, support tickets, emails and renewal chats. Some of it comes from a customer paying $120k a year, some from a customer paying $4k. Some say the same thing in completely different words. The PM's job is to decide **what to build next** and to **defend that decision** to engineers, designers and executives.

Rationale does the boring first 80% of that job: it reads everything, groups comments by the *need underneath*, adds up the revenue riding on each need, and ranks them. Then it writes the first draft of the spec, with every claim traced back to a real customer comment.

> **WHY THIS PROJECT EXISTS**
> It is a concept project built to show how a PM thinks: a clear user, a clear job, a simple and explainable score, and AI that is used where it helps and fenced in where it could do harm. The feedback in the sample is invented. No real customer data is involved.

The one-sentence pitch

"Paste your customer feedback and get a ranked, evidence-backed answer to 'what should we build next?', plus a spec you can trust, because every line of it is checked against what customers actually said."

What makes it different from "just asking a chatbot"

Plain chatbot	Rationale
Sorts by whatever sounds important	Sorts by one visible formula: revenue at stake x confidence / effort
Can invent customers and numbers	Specs are checked for invented customer ids and dollar figures
You cannot see why something ranked first	Every theme shows its formula and the comments behind it
Stops working when the AI is down	Falls back to simple word-overlap rules and a template spec
You paste private data straight to a model	Emails, phone numbers and links are stripped before any model call

CHAPTER 02 · WHY ANYONE NEEDS THIS

The problem, in plain words

ANALOGY

Imagine a restaurant with a box of comment cards. One says "waiting forever", another says "nobody brought water", a third says "table sat empty for 20 minutes". Different words, one need: **faster service**. A good manager reads the pile and says "service speed is our top problem". Rationale is that manager, for software feedback.

Why feedback is hard to use

- **Same need, different words.** "Approvals stall when the approver is on leave" and "we need escalation when nobody answers" are one problem.
- **Not all voices weigh the same.** A comment from a $120k enterprise account that says "we cannot renew without this" is not equal to a nice-to-have from a $5k account.
- **Effort is invisible in the comment.** The loudest request might take two days or two quarters.
- **Decisions need receipts.** A PM who says "trust me" loses the room. A PM who says "three accounts, $204k, here are their words" wins it.

The three questions a PM really asks

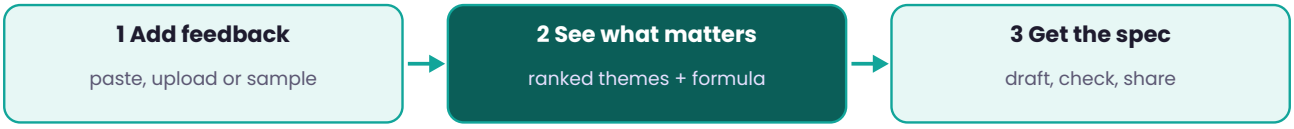
Question	How Rationale answers it
What are customers actually asking for?	Groups comments into 3 to 8 themes by the need underneath
Which of those matters most to the business?	Adds up revenue (ARR) per theme and ranks with a visible score
What do I write down to get it built?	Drafts a one-page spec and checks it against the evidence

PM LENS

Notice what Rationale does **not** try to do: it does not decide for you, and it does not claim to predict the future. It organises evidence and makes your reasoning visible. You can change effort and confidence with two buttons and watch the ranking re-order. That is the right level of ambition for a first version.

CHAPTER 03 · WHAT THE USER SEES

A walk through the product



Step 1: Add feedback

You can bring feedback in three shapes, and the app works out which one you used:

Shape	Example	What happens
CSV with a header row	customer, tier, arr, text	Columns are found by name, in any order, extra columns ignored
Pipe-separated lines	Northfield Bank | Enterprise | $120k | Need SSO	Customer, tier, ARR and comment are read from the four parts
Plain lines	Importing contracts one by one is painful	Each line is a comment; customer shows as Unknown with ARR 0 for you to fill in

ARR can be typed as **$84k**, **84**, **84,000** or **1.2m**. The app converts everything to thousands of dollars. There is also a **Load sample** button with invented customers like Acme Logistics, Brightwave and Northfield Bank, so anyone can try it in ten seconds.

> **WHY IT IS BUILT LIKE THIS**
> People do not have clean data. A tool that demands a perfect spreadsheet loses most users on the first screen. Accepting three messy shapes is a deliberate product choice: **meet the user's data where it is**. When ARR is missing, the app says so plainly, because ARR drives the ranking.

Step 2: See what matters

Rationale returns ranked **themes**. Each card shows the theme title, a one-line reason, how many customers are behind it, the revenue at stake, the comments themselves, and two small controls: **Effort** (Low, Medium, High) and **Confidence** (Low, Medium, High). Under each theme you see the maths written out, for example `Priority 45 = $200k x 0.9 / 4`.

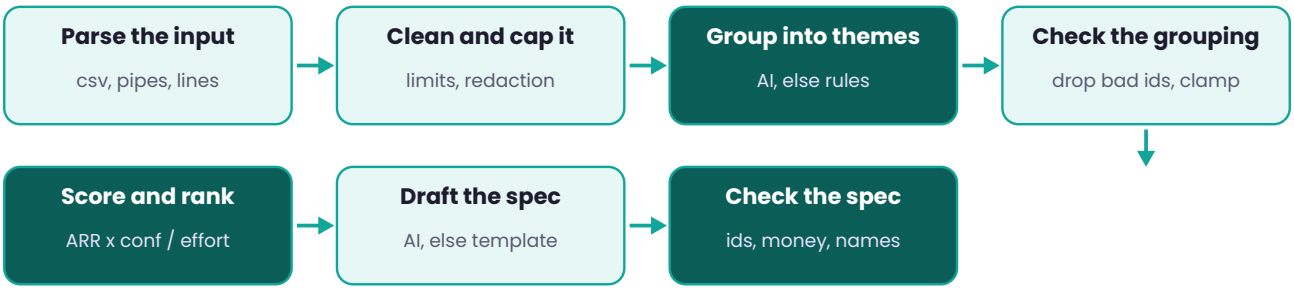
Change effort or confidence and the list re-sorts with a smooth animation. You can also export the ranking as a CSV (rank, theme, customers, ARR, confidence, effort, priority).

Step 3: Get the spec

Pick a theme and you get a one-page spec: problem, who is affected, evidence, goals, requirements, success metrics and risks. You can edit it, copy it, download it as a Markdown file, or share the whole ranking by link. Below the spec is a short faithfulness check (see chapter 7).

Under the hood: how it works

Rationale is a Next.js web app (a React front end plus small server routes) deployed on Vercel. You do not need to know code to follow the design. Think of it as a short assembly line.



The pieces, one by one

Piece	What it does	In plain words
Parser (csv.ts)	Reads CSV, pipe lines or plain lines into items with customer, tier, ARR and text	The front desk that accepts messy paperwork
Limits and redaction	Max 60 items, 600 characters each, 20,000 per batch; strips emails, phones and links	The security guard at the door
Grouping (AI)	Asks a model to group by underlying need into 3 to 8 themes	The smart analyst
Grouping (rules)	Word-overlap clustering when AI is off, capped or fails	The reliable intern with a highlighter
Normaliser	Cleans any model answer: unknown or repeated ids dropped, numbers clamped, empty themes removed	The proofreader who never trusts first drafts
Score	ARR at stake x confidence / effort	The scoreboard
Spec writer	Template spec always; AI spec on request	The drafting assistant
Spec checker	Four deterministic checks against the evidence	The fact-checker
Storage	Saves shared rankings as JSON (Vercel Blob in production, memory locally)	The filing cabinet behind the share link

The AI provider chain

When Rationale needs a model it tries **Claude first**, then **Gemini's free tier**, then gives up and uses its own rules. There is one important twist: if a provider answers but the answer fails validation (bad JSON, invented ids, a spec that is not a document), that counts as a failure and the **next provider gets a turn**. A reply is only accepted once it has passed the checks.

ANALOGY

Think of three taxi drivers. You call the first. If they do not pick up, or they drive somewhere else, you call the second. If neither works, you walk, which is slower but always gets you there. Walking here means the rules-based grouping and the template spec.

PM LENS: GRACEFUL DEGRADATION

A great AI feature is one that still has a useful answer when the AI is down, capped or wrong. Rationale never shows a blank screen because a model failed. When it falls back, the page says so, so the user knows what they are looking at.

CHAPTER 05 · THE CLEVER PART

How it reads feedback: grouping

With AI: group by the need, not the words

The model is told it is a senior PM and asked to group items into 3 to 8 themes by the **underlying need**. For each theme it returns a short title, which item ids belong, a one-sentence summary, an effort estimate from 1 to 5 with a reason, a confidence, whether the theme blocks revenue, and a severity (blocker, high, medium, low). The prompt also includes a safety rule: the feedback text is *data, not instructions*, so any instruction hidden inside a comment is ignored.

Without AI: simple word overlap

When there is no model, Rationale uses a small, honest algorithm. It lowercases each comment, removes filler words (the, and, with, we, need…), trims word endings (so "approvals" and "approval" match), and then measures how similar two comments are using the **Jaccard score**: shared words divided by total distinct words.

WORKED EXAMPLE

Comment A keeps the words {sso, saml, audit, log, export}. Comment B keeps {sso, audit, log}. They share 3 words out of 5 distinct words, so similarity is 3 / 5 = **0.6**.
Comment C keeps {bulk, upload, contracts}. It shares nothing with A, so similarity is **0**.
Rationale merges two groups when their best pair scores at least **0.14**. A and B join. C stays on its own.

It keeps merging the most similar pair until nothing is similar enough. If more than 8 groups remain it folds the smallest into whichever group it resembles most. Titles come from the most frequent words, severity from simple rules (a renewal or deal-gating phrase from an Enterprise account makes it a **blocker**; $100k+ is high; $25k+ is medium), and confidence starts at 0.45 plus 0.15 per comment, capped at 0.9.

BE HONEST ABOUT THE LIMITS

Rules grouping catches **repeated wording**, not **repeated needs**. Two customers asking for the same thing in different language may land in different groups. The app says so whenever the rules path is used. This is the main reason the AI path exists.

Never trust the first draft: the normaliser

Models can be creative in the wrong places. So every grouped answer goes through a normaliser before the user sees it:

- Item ids that do not exist are dropped, and an item can only appear in one theme.
- Numbers are clamped: effort to 1 to 5, confidence to 0.2 to 1. A model cannot return effort 47.
- Text is trimmed to sensible lengths.
- A theme with no valid evidence disappears. If nothing is left, the answer is rejected and the next provider (or the rules) takes over.

CHAPTER 06 · ONE LINE YOU CAN DEFEND

The priority score

Priority = revenue at stake x confidence / effort.

That is the entire formula. It is deliberately small so a PM can say it out loud in a meeting and a skeptical engineer can check it on a napkin.

Part	What it means	Where it comes from
Revenue at stake	Sum of ARR of every customer who raised this theme, in $ thousands	Your data (the ARR column)
Confidence	How sure are we this need is real? 0.2 to 1. Low, Medium, High buttons	AI proposes, you adjust. Rules path: 0.45 + 0.15 per comment, max 0.9
Effort	How big is the work? 1 to 5. Low, Medium, High buttons	AI proposes with a reason, you adjust. Rules path starts at 3

A worked example (numbers invented for teaching)

Theme	ARR at stake	Confidence	Effort	Priority
SSO and audit-log export	$200k	0.9	4	200 x 0.9 / 4 = **45**
Bulk contract import	$60k	0.7	2	60 x 0.7 / 2 = **21**
Approver delegation	$84k	0.6	3	84 x 0.6 / 3 = **16.8**
Dark mode	$10k	0.6	1	10 x 0.6 / 1 = **6**

Notice the story: SSO is big, but it is also expensive (effort 4). Bulk import has less revenue but is quick. The score balances both. If an engineer says "SSO is actually effort 5", change one button: 200 x 0.9 / 5 = 36, and the ranking updates live.

PM LENS: WHY A SIMPLE SCORE BEATS A FANCY ONE

A black-box score cannot be defended. A simple score gets **argued with**, which is what you want: people argue about confidence and effort (good conversation) instead of about whether the maths is magic.

This is a cousin of well-known frameworks like RICE (reach, impact, confidence, effort). Rationale swaps reach and impact for one concrete, checkable number: **ARR**. That is a trade-off: it favours revenue over strategy, so a strategic bet with little current ARR will under-rank. The PM stays in charge.

WATCH OUT

The score is only as good as the ARR and effort inputs. If ARR is missing (plain lines show Unknown with 0), those comments add no weight. The app nudges you to fill them in.

Writing the spec you can trust

Two ways to get a spec

	Template spec (no AI)	Draft with AI
When	Always available, instant	On request, if an AI key is set and budget remains
Sections	Problem, Evidence, Impact, Effort, Next step	Title, Problem, Who is affected, Evidence, Goals, Non-goals, Requirements (P0/P1), Success metrics, Risks and open questions
Length	Short	Under about 350 words, plain language
Evidence	Every comment listed as F-03 style ids with customer, ARR and tier	Must cite ids like F-03 and may not invent customers, numbers or deadlines

The four faithfulness checks

After an AI spec is written, four cheap, deterministic checks run. No second AI is involved, which makes them predictable and free. The page shows one calm line if everything passes and lists what to double-check if something does not.

Check	What it catches
Cites at least one feedback item	A spec with no evidence at all
Only cites items from this theme	Mentioning F-17 when F-17 belongs to a different theme
No dollar figures outside the evidence	Invented money: "$2M opportunity" when the evidence says $204k
Names the customers it relies on	A spec that talks about "customers" in the abstract

ANALOGY

Think of an intern who writes a report and must footnote every claim. A manager does not re-do the research; they just check that each footnote points at a real page. If a footnote points nowhere, the claim is suspect. The four checks are that footnote audit.

PM LENS: TRUST IS A FEATURE

In AI products, "how do I know it did not make this up?" is the first question every sceptical stakeholder asks. Rationale answers it in the interface, not in a disclaimer. When you present a spec you can say: *every figure here traces to a customer comment, and the tool checked that for me.*

Safe display

Model output is shown through a tiny Markdown renderer that **escapes everything first**. Even if a model tried to slip in a script tag, it would appear as harmless text. Small detail, big difference in security.

CHAPTER 08 · A PUBLIC DEMO NEEDS GUARDRAILS

Safety, privacy and abuse protection

Rationale is a public demo anyone can hit, and AI calls cost real money. So it is defended in layers, like a building with a lobby, a badge reader and a locked server room.

Layer	What it does	Why it matters
Input caps	60 items, 600 characters each, 20,000 characters per batch	Stops huge pastes from burning budget
Schema validation (zod)	Every request must match a strict shape	Rejects malformed or hostile input early
Redaction	Emails, phone numbers and links become [email], [phone], [link] before any model call	Keeps accidental personal data away from models
Prompt-injection stance	Prompts say feedback is data, not instructions	A comment saying "ignore all rules" is just a comment
Per-visitor rate limits	Hourly caps on grouping (8), spec drafts (12) and saves	Stops one person from draining the demo
Daily AI cap	A shared counter of model calls per day (default 400)	Hard ceiling on total spend
Fail-safe fallbacks	Rules grouping and template spec always work	Limits never create a dead end

HONEST LIMITS

Saved links are readable by anyone who has the link (like an unlisted video). Limits are tracked per server instance, which is enough to stop casual abuse but is not bank-grade. The README says so openly.

PM LENS: COST IS A PRODUCT REQUIREMENT

AI features have a marginal cost per click. A PM should ask: what is the cost per use, what stops runaway cost, and what does the user see when the cap is hit? Rationale has an answer to all three: rate limits, a daily cap, and a polite message plus the rules fallback.

Product decisions and trade-offs

Good PMs can explain not just what they built but what they chose **not** to build. Here are the main decisions in Rationale, with the reasoning you could give in an interview.

Decision	Why	What we gave up
One visible formula for priority	Explainable and arguable	Nuance (strategy, dependencies, risk)
ARR as the weight	Concrete, available, defensible to finance	Strategic bets with little current revenue under-rank
Effort and confidence as three-level buttons	Fast and honest. False precision is worse than none	Fine-grained estimates
AI proposes, human adjusts	PM keeps ownership of the decision	Full automation
Rules fallback instead of an error	Always useful, demo never breaks	Lower grouping quality on that path
Deterministic spec checks, not an AI judge	Cheap, repeatable, explainable	Cannot catch subtle wrong reasoning, only invented ids and money
Accept messy input shapes	Low friction on the first screen	More parsing code and edge cases
Share by link, no accounts	Zero sign-up friction for a demo	Anyone with the link can read it
Three steps, one screen each	Clear mental model, mobile friendly	Power-user shortcuts

What a real product would add next

- Connect to sources (Zendesk, Intercom, Gong, Slack) so feedback arrives by itself.
- Pull ARR from the CRM automatically instead of typing it.
- Track themes over time: is "SSO" getting louder this quarter?
- Link a theme to a ticket in Jira or Linear when you commit to it.
- A calibration set: does the AI grouping match how real PMs group? Today this is a judgement call, not a measured score.

Metrics, quality and testing

Product metrics you could track

Stage	Metric	What it tells you
Activation	% of visitors who load sample or paste data	Is the first screen clear?
Core value	% who reach the ranked list; % who open a spec	Did they get to the 'aha'?
Engagement	Effort and confidence edits per session	Are PMs using the score as a thinking tool?
Output	Specs copied, downloaded or shared	Was the output good enough to use?
Quality	Spec check pass rate; AI fallback rate	How often does the AI earn trust?
Cost	Model calls per active user; days the cap is hit	Is it sustainable?

How it is tested

There is a small automated test suite (8 groups of tests) that runs on every change. It checks that: the score ranks security-gating themes first on the sample; rules grouping puts the SSO and audit-log comments together and keeps every item exactly once; CSV and line parsing handle quotes and odd shapes; model output is cleaned correctly; the spec checker catches invented ids and money; Markdown display escapes markup; redaction and input limits hold; and JSON can be pulled out of a chatty model reply.

> **PM LENS**
> Notice that the things tested are the **trust boundaries**: where untrusted input (user text, model output) meets the product. That is where AI products fail, so that is where tests belong.

Interview cheat sheet

The 30-second version

"I built Rationale to answer the PM question: what should we build next, and can I prove it? You paste customer feedback; it groups comments by the need underneath, ranks them by revenue at stake times confidence over effort, and drafts a spec where every claim is checked against the customer evidence. The AI is fenced in: if it fails, simple rules take over, and a spec can never cite a customer or a dollar figure that is not in the feedback."

The 2-minute version, in four beats

1. **Problem.** PMs drown in feedback that uses different words for the same need and gives every voice equal weight.
2. **Solution.** Three screens: add feedback, see ranked themes with the formula visible, get a checked spec.
3. **Design choices.** A one-line score anyone can argue with; AI proposes while the PM adjusts; deterministic checks instead of trusting the model; graceful fallback.
4. **Honest limits and next steps.** Rules grouping catches wording not meaning; ARR-weighting under-ranks strategic bets; next I would connect real feedback sources and measure grouping quality against PM judgement.

Likely questions and good answers

Why not just ask ChatGPT?

A chat gives you prose you cannot audit. Rationale gives a structured ranking with a visible formula, the underlying comments, and a spec checked against evidence. The value is the workflow plus the trust checks, not the text generation.

How do you stop the AI from making things up?

Three layers. The prompt says never invent facts. The normaliser drops unknown ids and clamps numbers. After a spec is written, four deterministic checks look for invented ids, invented dollar figures and missing customer names, and the page shows the result.

Why is your score so simple?

So it can be explained and challenged. People argue about the inputs, confidence and effort, which is the conversation a PM wants. A complex score hides the argument. The trade-off is that it ignores strategy and dependencies, so it informs the decision and does not make it.

What if the AI is down or you run out of budget?

Grouping falls back to word-overlap rules and the spec to a template. The app says which path it used. There is also a daily cap and per-visitor limits so the demo cannot be drained.

What would you measure?

Activation (sample or paste), reaching the ranking, specs copied or shared, effort and confidence edits as a sign people use the score, spec-check pass rate, AI fallback rate and model calls per user.

What would you do next?

Integrations for feedback and ARR, trends over time, a link from theme to ticket, and a labelled set to measure grouping quality against how real PMs group.

What is the biggest weakness?

Grouping quality on the rules path, and weighting by ARR. I would say both openly, because knowing the limits is part of the product.

How is privacy handled?

Emails, phone numbers and links are stripped before any model call, inputs are capped, and the feedback text is treated as data, not instructions. Shared links are readable by anyone with the link, which is stated in the README.

CHAPTER 12 · REFERENCE

Glossary and code map

Glossary

Term	Meaning
ARR	Annual recurring revenue: what a customer pays per year. Rationale stores it in $ thousands
Theme	A group of comments that share one underlying need
Tier	Enterprise, Mid-market or SMB: the size band of the customer
Effort / Confidence	How big the work is (1 to 5) and how sure we are the need is real (0.2 to 1)
Blocker	A theme where a renewal, expansion or deal depends on it
Jaccard similarity	Shared words divided by total distinct words between two comments
Normaliser	Code that cleans a model's answer before the app uses it
Provider chain	The order of AI services tried: Claude, then Gemini, then rules
Prompt injection	Hidden instructions inside user text that try to steer a model
Redaction	Removing personal data (email, phone, links) before text leaves your app
Graceful degradation	Still being useful when a part of the system fails
Blob store	Simple cloud file storage, used here for shared rankings

Where things live in the code

Repository: `apps/rationale` in the portfolio repo.

File	What is inside
lib/csv.ts	Parsing CSV, pipe lines, plain lines; ARR and tier reading
lib/cluster.ts	Rules grouping: word cleaning, Jaccard, merging, titles, severity
lib/score.ts	The priority formula, ranking, ids like F-03
lib/validate.ts	Input schemas and the normaliser for model output
lib/prd.ts	Template spec, the four spec checks, safe Markdown display
lib/ai.ts	Claude then Gemini then fail; JSON extraction
lib/limit.ts, redact.ts	Rate limits, daily AI cap, personal-data stripping
lib/store.ts	Saving shared rankings
app/api/group, prd, sessions	The three server routes
components/Workspace.tsx	The three-step screen
tests/core.test.ts	The automated tests